Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение высшего образования
«Московский государственный технический университет имени Н. Э. Баумана (национальный исследовательский университет)»
(МГТУ им. Н. Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления» (ИУ)
КАФЕДРА «Информационная безопасность» (ИУ8)

Отчёт

по научно-исследовательской работе студента

на тему Классификатор адресов в сети криптовалюты Bitcoin

ФИО студента: Санджиев Темирхан Санджиевич

Группа: ИУ8-93-2024

Специальность: 10.05.03 «Информационная безопасность автоматизированных систем

Специализация: 10.05.03_03 «Создание автоматизированных систем в защищенном ис

Научный руководитель НИРС: профессор кафедры ИУ8 Зуев Юрий Анатольевич

Работа выполнена:	_____	_____	Санджиев Т. С.
	Дата	Подпись	(И. О. Фамилия)
Допуск к защите:	_____	_____	Зуев Ю. А.
	Дата	Подпись	(И. О. Фамилия)

Дата защиты НИРС: _____

Результаты защиты: _____

Москва, 2026 г.

РЕФЕРАТ

Отчёт содержит 43 стр., 15 рис., 13 источн.В рамках работы разработан и апробирован программный комплекс для классификации Bitcoin-адресов по типу кошелька на основе датасета, сформированного в ходе предшествующих НИРС. Реализован полный пайплайн обработки данных: удаление высококоррелированных признаков, автоматический выбор стратегии устранения дисбаланса классов (SMOTE против ADASYN), стандартизация. Обучено шесть моделей машинного обучения: MLP, Random Forest, XGBoost, LightGBM, Stacking и kNN. Для деревьевидных моделей выполнена настройка гиперпараметров методом случайного поиска с кросс-валидацией. Проведена комплексная оценка качества: построены матрицы ошибок, ROC-кривые, вычислены взвешенная F1-мера и макро-усреднённый ROC-AUC. Для лучшей модели выполнена оптимизация порогов классификации по критерию Юдена и интерпретация предсказаний методом SHAP. Наилучший результат достигнут моделью XGBoost: взвешенная F1-мера — 0,7907; ROC-AUC — 0,9610.

Ключевые слова: Bitcoin, блокчейн, классификация адресов, машинное обучение, XGBoost, LightGBM, нейронная сеть, Stacking, SMOTE, SHAP.

СОДЕРЖАНИЕ

РЕФЕРАТ . . . 4
ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ . . . 6
СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ . . . 8
ВВЕДЕНИЕ . . . 9
1 Обучение моделей классификации на данных, полученных в предыдущих НИРС . . . 11
1.1 Подготовка данных к обучению . . . 11
1.1.1 Отбор признаков и устранение мультиколлинеарности . . . 11
1.1.2 Разбиение выборки . . . 12
1.1.3 Устранение дисбаланса классов . . . 12
1.1.4 Стандартизация признаков . . . 13
1.2 Обучение MLP . . . 14
1.3 Ансамблевые методы . . . 15
1.3.1 Random Forest . . . 15
1.3.2 XGBoost . . . 16
1.3.3 LightGBM . . . 17
1.3.4 Stacking . . . 17
1.3.5 kNN . . . 18
1.4 Настройка гиперпараметров . . . 18
1.5 Программная реализация . . . 19
2 Анализ результатов обучения моделей . . . 21
2.1 Используемые метрики оценки качества . . . 21
2.2 Сводные результаты сравнения моделей . . . 22
2.3 Матрицы ошибок . . . 22
2.4 ROC-кривые и оптимизация порогов классификации . . . 29
2.4.1 Анализ ROC-кривых . . . 29
2.4.2 Критерий Юдена и оптимизация порогов . . . 35
2.5 Интерпретация предсказаний методом SHAP . . . 36
2.6 Сравнительный анализ моделей . . . 39
2.7 Анализ проблемных классов . . . 40
ЗАКЛЮЧЕНИЕ . . . 41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ . . . 42

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящем отчете о НИР применяют следующие термины с соответствующими определениями:

Адрес — Уникальный идентификатор в сети Bitcoin, используемый для отправки и получения средств

Батч — Подмножество обучающих данных, используемое для одного шага обновления весов нейронной сети

Блокчейн — Распределённая база данных, состоящая из последовательно связанных блоков, содержащих информацию о транзакциях

Гиперпараметр — Параметр модели или процесса обучения, задаваемый до начала обучения и не изменяемый в его ходе

Граф транзакций — Графовое представление Bitcoin-сети, где узлами являются адреса, а рёбрами — транзакции между ними

Датасет — Набор данных, используемый для обучения, тестирования и валидации алгоритмов машинного обучения

Дропаут — Метод регуляризации нейронных сетей, при котором случайные нейроны отключаются во время обучения для предотвращения переобучения

Извлечение признаков — Процесс выделения характеристик из данных для использования в алгоритмах машинного обучения

Классификация — Задача машинного обучения, заключающаяся в отнесении объектов к предопределённым классам

Кросс-валидация — Метод оценки качества модели путём разбиения данных на несколько частей и поочерёдного использования каждой из них в качестве тестовой

Матрица ошибок — Таблица, отображающая соотношение предсказанных и истинных классов, используемая для оценки качества классификации

Нейронная сеть — Вычислительная модель, вдохновлённая биологическими нейронными сетями, состоящая из слоёв взаимосвязанных узлов, обучаемых на данных

Обучение модели — Процесс настройки параметров нейронной сети путём минимизации функции потерь на обучающей выборке

Оптимизатор — Алгоритм, обновляющий параметры модели на основе вычисленных градиентов с целью минимизации функции потерь

Переобучение — Явление, при котором модель слишком точно подстраивается

под обучающую выборку и теряет способность к обобщению

Транзакция — Операция передачи Bitcoin между адресами, записанная в блокчейне

Функция потерь — Функция, измеряющая расхождение между предсказаниями модели и истинными метками, минимизируемая в процессе обучения

Эпоха — Один полный проход по всему обучающему датасету в ходе обучения нейронной сети

Bitcoin — Криптовалюта и одноимённая платёжная система, использующая технологию блокчейн для децентрализованного учёта транзакций

UTXO — Unspent Transaction Output — модель учёта средств в Bitcoin, где каждый выход транзакции может быть потрачен только один раз

СПИСОК ОБОЗНАЧЕНИЙ И СОКРАЩЕНИЙ

В настоящем отчете о НИР применяют следующие сокращения и обозначения:

ADASYN — Adaptive Synthetic Sampling
AML — Anti-Money Laundering
API — Application Programming Interface
AUC — Area Under the Curve
BTC — Bitcoin
CPU — Central Processing Unit
CSV — Comma-Separated Values
DL — Deep Learning
F1 — F1-мера
GAT — Graph Attention Network
GCN — Graph Convolutional Network
GNN — Graph Neural Network
GPU — Graphics Processing Unit
JSON — JavaScript Object Notation
KYC — Know Your Customer
LN — Lightning Network
LSTM — Long Short-Term Memory
ML — Machine Learning
MLP — Multilayer Perceptron
NN — Neural Network
PCA — Principal Component Analysis
PoW — Proof of Work
RAM — Random Access Memory
ReLU — Rectified Linear Unit
ROC — Receiver Operating Characteristic
SegWit — Segregated Witness
SHAP — SHapley Additive exPlanations
SMOTE — Synthetic Minority Over-sampling Technique
TXID — Transaction Identifier
UTXO — Unspent Transaction Output

ВВЕДЕНИЕ

Криптовалюта Bitcoin, созданная в 2008 году Сатоши Накамото [1], продолжает оставаться наиболее капитализированной и широко используемой децентрализованной платёжной системой. В её основе лежит технология распределённого реестра — блокчейна — представляющего собой публично доступную цепочку блоков транзакций, подтверждённых майнерами посредством алгоритма PoW. Публичная природа блокчейна обеспечивает полную верифицируемость транзакций, однако участники сети идентифицируются лишь псевдонимными адресами — хешами открытых ключей — без привязки к реальным личностям. Это создаёт серьёзные вызовы для регуляторов, правоохранительных органов и аналитиков финансовых рисков, заинтересованных в выявлении подозрительной активности: отмывания денег, мошенничества, финансирования незаконной деятельности.

Классификация Bitcoin-адресов по типу деятельности — задача, решение которой позволяет частично снять псевдоанонимность участников сети. Если известно, что некоторый адрес принадлежит бирже или майнинговому пулу, аналитик может интерпретировать транзакции с этим адресом и реконструировать финансовые потоки. В отличие от ручного анализа или эвристических методов, основанных на фиксированных правилах, подходы на основе машинного обучения позволяют автоматически выявлять сложные паттерны в пространстве признаков, масштабироваться на большие объёмы данных и адаптироваться к новым видам активности.

В ходе предшествующих научно-исследовательских работ (НИРС-01 и НИРС-02) были решены задачи систематизации существующих алгоритмов классификации транзакций, сбора Bitcoin-адресов из открытых источников, извлечения 34 транзакционных признаков через API Bitcoin-эксплорера и формирования датасета объёмом 8 115 адресов с разметкой по шести типам кошельков. Выявлена существенная несбалансированность классов (коэффициент $\approx 39{,}85$) и определён набор наиболее информативных признаков.

Настоящая работа является третьим этапом исследовательского цикла и посвящена практическому применению подготовленного датасета: обучению, настройке гиперпараметров и сравнительной оценке моделей классификации Bitcoin-адресов, а также интерпретации полученных результатов.

Актуальность работы обусловлена:

– ростом объёма криптовалютных транзакций и потребностью в масштабируемых автоматизированных инструментах мониторинга — ручной анализ блокчейна в современных масштабах практически невозможен;
– регуляторными требованиями соблюдения процедур AML/KYC в криптовалютной сфере, которые обязывают биржи и сервисы идентифицировать контрагентов;
– научным интересом к применению ансамблевых методов машинного обучения для анализа высокодимензиональных блокчейн-данных с дисбалансом классов;
– необходимостью сравнительного анализа нейросетевых и деревьевидных подходов на табличных Bitcoin-данных.

Целью данной научно-исследовательской работы является разработка, обучение и сравнительная оценка ансамблевых и нейросетевых моделей классификации Bitcoin-адресов по типам кошельков на основе датасета, подготовленного в предшествующих НИРС, а также анализ качества и интерпретируемости полученных результатов.

Для достижения цели поставлены следующие задачи:
– Подготовка признакового пространства: удаление высококоррелированных признаков, выбор и применение стратегии устранения дисбаланса классов, стандартизация данных;
– Обучение шести моделей машинного обучения: MLP, Random Forest, XGBoost, LightGBM, Stacking и kNN;
– Настройка гиперпараметров деревьевидных моделей методом случайного поиска с кросс-валидацией;
– Оценка качества обученных моделей с использованием метрик взвешенной F1-меры и макро-усреднённого ROC-AUC на отложенной тестовой выборке;
– Оптимизация порогов классификации лучшей модели по критерию Юдена — для улучшения полноты на миноритарных классах;
– Интерпретация предсказаний лучшей модели методом SHAP для выявления наиболее значимых признаков;
– Сравнительный анализ всех обученных моделей и формулировка рекомендаций по дальнейшему улучшению.

1 Обучение моделей классификации на данных, полученных в предыдущих НИРС

1.1 Подготовка данных к обучению

Исходным материалом для данного этапа служит датасет, сформированный в рамках НИРС-02 [2]. Датасет содержит 8 115 Bitcoin-адресов, каждый из которых описывается 34 признаками. Признаки сгруппированы по смыслу:

- *структурные* — число входов и выходов транзакций, количество уникальных контрагентов, средняя вентиляторность (fan-in/fan-out);
- *UTXO-паттерны* — доля нераспределённых выходов, средний остаток на адресе, характеристики типичного изменения UTXO;
- *временны́е* — общий период активности адреса, средний интервал между транзакциями, число активных дней;
- *экономические* — суммарный объём входящих и исходящих средств (в satoshi), средний объём одной транзакции, соотношение объёмов;
- *Bitcoin-специфичные* — наличие SegWit-выходов, участие в мультиподписных схемах, паттерны округления сумм.

Целевая переменная — тип кошелька — принимает шесть значений: майнеры (*miner*, 3 547 адресов, 43,7%), биржи (*exchange*, 1 532, 18,9%), сервисы (*services*, 1 249, 15,4%), азартные игры (*gambling*, 911, 11,2%), операции типа CoinJoin (*coinjoin-like*, 787, 9,7%) и майнинговые пулы (*mining_pool*, 89, 1,1%).

1.1.1 Отбор признаков и устранение мультиколлинеарности

Наличие высококоррелированных признаков в обучающей выборке нежелательно: оно не несёт дополнительной информации, но увеличивает размерность пространства, замедляет обучение и может нестабилизировать оценки важности признаков. Степень линейной зависимости между парой непрерывных признаков x_i, x_j измеряется коэффициентом корреляции Пирсона:

$$r(x_i, x_j) = \frac{\sum_{k=1}^{n}(x_i^{(k)} - \bar{x}_i)(x_j^{(k)} - \bar{x}_j)}{\sqrt{\sum_{k=1}^{n}(x_i^{(k)} - \bar{x}_i)^2 \cdot \sum_{k=1}^{n}(x_j^{(k)} - \bar{x}_j)^2}},$$

где $x_i^{(k)}$, $x_j^{(k)}$ — значения признаков x_i и x_j для k-го образца; $\bar{x}_i$, $\bar{x}_j$ — их выборочные средние; n — число образцов. Если $|r(x_i, x_j)| > 0{,}95$, один из признаков удаляется. По результатам автоматического анализа корреляционной матрицы исключены четыре признака: *unique_input_addresses* и *unique_output_addresses* — как практически линейно зависимые от агрегированных счётчиков входов и выходов, — а также два дополнительных признака, выявленных алгоритмом попарной проверки. Итоговое признаковое пространство составляет 30 переменных.

1.1.2 Разбиение выборки

Датасет разбивается на три непересекающихся подмножества: обучающую выборку (70%), валидационную (15%) и тестовую (15%). Разбиение выполняется стратифицированно — пропорции классов сохраняются во всех трёх частях. Это критически важно при дисбалансе: нестратифицированное разбиение может случайно недопредставить миноритарный класс (*mining_pool*, 89 образцов) в тестовой выборке, что сделает оценку качества нестабильной. После стратификации в тестовой выборке находится $\lceil 89 \cdot 0{,}15 \rceil = 14$ адресов класса *mining_pool*.

Важно, что оверсэмплинг и нормализация применяются исключительно к обучающей выборке, тогда как валидационная и тестовая выборки остаются в исходном виде — это исключает утечку данных (data leakage) и обеспечивает объективную оценку обобщающей способности модели.

1.1.3 Устранение дисбаланса классов

Дисбаланс классов с коэффициентом $\approx$ 39,85 (отношение числа образцов мажоритарного класса к миноритарному) является существенной проблемой: классификатор, обученный на несбалансированных данных, стремится максимизировать суммарную точность за счёт занижения предсказаний для редких классов. Для устранения дисбаланса рассматриваются два метода оверсэмплинга:

SMOTE (Synthetic Minority Over-sampling Technique) [3] — для каждого образца миноритарного класса $\mathbf{x}_i$ выбирается k ближайших соседей того же класса (по умолчанию $k = 5$). Синтетический образец $\tilde{\mathbf{x}}$ формируется интерполяцией:

$$\tilde{\mathbf{x}} = \mathbf{x}_i + \lambda \cdot (\mathbf{x}_{nn} - \mathbf{x}_i), \quad \lambda \sim U(0, 1),$$

где $\tilde{\mathbf{x}}$ — синтетический образец; $\mathbf{x}_i$ — исходный образец миноритарного класса; $\mathbf{x}_{nn}$ — случайно выбранный из k ближайших соседей образец того же класса; $\lambda \sim U(0,1)$ — коэффициент интерполяции. Такой подход создаёт разнообразные, но не выходящие за пределы выпуклой оболочки класса образцы.

ADASYN (Adaptive Synthetic Sampling) — адаптивная модификация SMOTE, при которой интенсивность генерации для каждого образца пропорциональна плотности ошибок в его окрестности: больше синтетических примеров генерируется там, где класс труднее всего отделить от мажоритарных. Формально для образца $\mathbf{x}_i$ вычисляется $r_i = \Delta_i / K$, где Δ_i — число соседей из мажоритарного класса среди K ближайших соседей образца $\mathbf{x}_i$ — доля таких соседей; число синтетических образцов для $\mathbf{x}_i$ пропорционально $\hat{r}_i = r_i / \sum_j r_j$, где суммирование ведётся по всем образцам миноритарного класса.

Выбор между SMOTE и ADASYN производится автоматически: прокси-модель Random Forest (50 деревьев) обучается раздельно на каждом варианте оверсэмплинга и оценивается по взвешенной F1-мере на оригинальной (не синтетической) валидационной выборке. По результатам сравнения выбран SMOTE (val F1 = 0,7475 против 0,7337 у ADASYN). Предпочтение SMOTE объясняется тем, что ADASYN, концентрируя генерацию в граничных зонах, увеличивает шум в разделяющей гиперплоскости — это может ухудшать обобщение. После применения SMOTE обучающая выборка расширяется до 14 892 образцов с равным представлением каждого класса.

Дополнительно модели Random Forest и LightGBM обучаются с параметром `class_weight="balanced"`, который масштабирует веса ошибок обратно пропорционально частоте класса. Это приводит к тому, что ошибка на одном образце класса *mining_pool* штрафуется примерно в 40 раз сильнее, чем ошибка на образце класса *miner*.

1.1.4 Стандартизация признаков

После оверсэмплинга признаки стандартизируются методом *StandardScaler*: каждый признак x_j преобразуется к нулевому среднему и единичному стандартному отклонению:

$$\hat{x}_j = \frac{x_j - \mu_j}{\sigma_j},$$

где x_j — исходное значение j-го признака; $\hat{x}_j$ — его стандартизованное значение; μ_j и σ_j — среднее и стандартное отклонение признака j, вычисленные по обучающей выборке [4]. Стандартизация необходима для MLP и kNN, поскольку оба метода чувствительны к масштабу признаков; для деревьевидных методов она не влияет на результат, однако сохраняется для унификации пайплайна.

1.2 Обучение MLP

Полносвязная нейронная сеть (MLP) рассматривается как основная нейросетевая модель в данной работе. MLP — это направленный граф слоёв нейронов: входной слой принимает вектор стандартизованных признаков размерности 30; далее следуют три скрытых слоя (256, 128 и 64 нейрона); выходной слой содержит 6 нейронов — по одному на класс — с функцией активации *Softmax*, преобразующей логиты в вектор вероятностей.

На каждом скрытом слое применяется функция активации ReLU:

$$\mathrm{ReLU}(z) = \max(0, z),$$

где z — взвешенная сумма входов нейрона. Функция обеспечивает нелинейность преобразования и устраняет проблему затухающих градиентов. Обучение производится методом обратного распространения ошибки: вычисляется градиент функции потерь по всем весам сети, после чего веса обновляются оптимизатором Adam [5] — адаптивным стохастическим градиентным спуском с моментом первого и второго порядка:

$$\theta_{t+1} = \theta_t - \frac{\eta}{\sqrt{\hat{v}_t} + \varepsilon} \cdot \hat{m}_t,$$

где θ_t — вектор параметров модели на шаге t; $\hat{m}_t$ и $\hat{v}_t$ — скорректированные на смещение оценки первого и второго момента градиента; η — скорость обучения; ε — малая константа для численной стабильности. В качестве функции потерь используется кросс-энтропия.

Для предотвращения переобучения применяется L2-регуляризация (*weight decay*) с коэффициентом $\alpha = 10^{-4}$: к функции потерь добавляется штрафной член $\alpha\|\mathbf{W}\|_F^2$, препятствующий росту весов. Дополнительно используется ранняя остановка (*early stopping*): если взвешенная F1-мера на валидационной доле обучающей выборки (10%) не улучшается на протяжении 10 итераций подряд, обучение прекращается и восстанавливаются веса с наилучшим значением метрики.

Гиперпараметры MLP зафиксированы по результатам предварительных экспериментов без использования автоматического поиска: объём обучающей выборки (14 892 образцов) в сочетании с высокой сложностью MLP делает перебор конфигураций через кросс-валидацию нецелесообразным по времени. Итоговые параметры: скорость обучения $5 \cdot 10^{-4}$, размер батча 32, $\alpha = 10^{-4}$, максимальное число итераций 200.

1.3 Ансамблевые методы

1.3.1 Random Forest

Random Forest [4] — ансамблевый метод, основанный на принципе бэггинга (*bootstrap aggregating*): для каждого из B деревьев формируется бутстрэп-подвыборка (с возвращением) из обучающей выборки, а при выборе оптимального разбиения в каждом узле рассматривается случайное подмножество признаков размером $\sqrt{p}$ или $\log_2 p$. Итоговый класс определяется мажоритарным голосованием деревьев. Случайность при отборе признаков снижает корреляцию между деревьями и повышает устойчивость ансамбля к переобучению.

Для борьбы с дисбалансом классов используется параметр `class_weight="balanced"`, при котором вес каждого класса c задаётся как $w_c = n/(K \cdot n_c)$, где n — полное число образцов, K — число классов, n_c — число образцов класса c.

Пространство поиска гиперпараметров:

- число деревьев B: $\{100, 200\}$;
- максимальная глубина дерева: $\{\text{без ограничения},\ 15\}$;
- минимальное число образцов для разбиения узла: $\{2,\ 5\}$;
- число признаков при разбиении: $\{\sqrt{p},\ \log_2 p\}$.

1.3.2 XGBoost

XGBoost [6] — реализация градиентного бустинга на деревьях решений. В отличие от бэггинга, бустинг строит деревья последовательно: каждое новое дерево h_t обучается аппроксимировать отрицательный градиент функции потерь $\mathcal{L}$ по предсказаниям текущего ансамбля. Итоговый ансамбль — аддитивная модель:

$$F_T(\mathbf{x}) = \sum_{t=1}^{T} \eta \cdot h_t(\mathbf{x}),$$

где $F_T(\mathbf{x})$ — итоговое предсказание ансамбля из T деревьев для объекта $\mathbf{x}$; $h_t(\mathbf{x})$ — предсказание t-го дерева; η — скорость обучения, масштабирующая вклад каждого дерева.

Ключевое отличие XGBoost — включение в целевую функцию регуляризационных членов, штрафующих сложность каждого дерева:

$$\widetilde{\mathcal{L}}^{(t)} = \sum_i l\big(y_i,\, \hat{y}_i^{(t)}\big) + \gamma T_t + \frac{1}{2}\lambda \|\mathbf{w}_t\|^2,$$

где $\widetilde{\mathcal{L}}^{(t)}$ — регуляризованная целевая функция на шаге t; $l(y_i, \hat{y}_i^{(t)})$ — функция потерь для i-го обучающего образца; y_i — истинная метка класса; $\hat{y}_i^{(t)}$ — текущее предсказание ансамбля; T_t — число листьев добавляемого дерева; $\mathbf{w}_t$ — вектор весов листьев; γ и λ — гиперпараметры регуляризации. Это предотвращает переобучение даже на глубоких деревьях.

Пространство поиска гиперпараметров:

- число деревьев T: $\{100, 200\}$;
- максимальная глубина дерева: $\{4,\ 6\}$;
- скорость обучения η: $\{0{,}1;\ 0{,}2\}$;
- доля случайно выбираемых образцов для дерева (*subsample*): $\{0{,}8;\ 1{,}0\}$;
- доля случайно выбираемых признаков (*colsample_bytree*): $\{0{,}8;\ 1{,}0\}$.

Параметры *subsample* и *colsample_bytree* вносят стохастичность, аналогичную бэггингу, и снижают корреляцию между деревьями.

1.3.3 LightGBM

LightGBM [7] — высокопроизводительная реализация градиентного бустинга, использующая два ключевых нововведения. Во-первых, *алгоритм листового роста*: вместо уровневого (depth-wise) наращивания дерева LightGBM на каждом шаге добавляет лист к узлу с наибольшим приростом функции. Это позволяет строить глубокие несимметричные деревья за меньшее число шагов. Во-вторых, *гистограммная аппроксимация*: непрерывные признаки дискретизируются в B бинов (по умолчанию 255), что снижает вычислительную сложность поиска оптимального разбиения с $O(n \cdot p)$ до $O(B \cdot p)$. В совокупности это обеспечивает скорость обучения в 10–100 раз выше, чем у XGBoost, при сопоставимом качестве.

Используется параметр `class_weight="balanced"`. Пространство поиска:

- число деревьев: $\{100, 200\}$;
- максимальная глубина дерева: $\{4,\ 6,\ -1\}$ (без ограничения);
- скорость обучения: $\{0{,}1;\ 0{,}2\}$;
- максимальное число листьев (*num_leaves*): $\{31,\ 63\}$;
- доля образцов для построения дерева (*subsample*): $\{0{,}8;\ 1{,}0\}$.

Параметр *num_leaves* является основным регулятором сложности в LightGBM: при листовом росте глубина дерева сама по себе не ограничивает сложность, поэтому ограничение числа листьев эквивалентно ограничению глубины в XGBoost.

1.3.4 Stacking

Stacking (*StackingClassifier*) — двухуровневый ансамбль. На первом уровне три базовых модели — Random Forest, XGBoost и LightGBM — обучаются на обучающей выборке. Для формирования признаков мета-уровня применяется трёхкратная стратифицированная кросс-валидация: обучающая выборка делится на три фолда; каждая базовая модель обучается на двух фолдах и предсказывает вероятности для третьего. Таким образом для каждого объекта обучающей выборки вычисляется вектор вероятностей без использования самого объекта в обучении — это исключает утечку данных. Итоговая матрица мета-признаков имеет размерность $n_{\text{train}} \times (3 \cdot K)$, где $K = 6$ — число классов.

На втором уровне мета-классификатор — логистическая регрессия — обучается на мета-признаках. При оценке на тестовой выборке базовые модели дообучаются на полной обучающей выборке и формируют мета-признаки для тестовых объектов; затем мета-классификатор выдаёт финальное предсказание.

1.3.5 kNN

kNN — непараметрический классификатор, не требующий явного обучения. Для классификации нового объекта $\mathbf{x}_{\text{test}}$ вычисляется евклидово расстояние до всех объектов обучающей выборки, отбираются k ближайших, и класс определяется большинством голосов:

$$\hat{y} = \arg\max_{c} \sum_{i \in N_k(\mathbf{x}_{\text{test}})} \mathbf{1}[y_i = c],$$

где $\hat{y}$ — предсказанный класс; c — перебираемый класс; $N_k(\mathbf{x}_{\text{test}})$ — множество индексов k ближайших соседей объекта $\mathbf{x}_{\text{test}}$; y_i — истинная метка i-го соседа; $\mathbf{1}[\cdot]$ — индикаторная функция. Используется $k = 7$ без дополнительной настройки — как простейший базовый классификатор. Метод чувствителен к масштабу признаков (поэтому необходима стандартизация) и к «проклятию размерности» в высокодимензиональных пространствах, однако хорошо иллюстрирует, насколько линейно не разделимы классы в исходном признаковом пространстве.

1.4 Настройка гиперпараметров

Для Random Forest, XGBoost и LightGBM применяется случайный поиск гиперпараметров (*RandomizedSearchCV*) [4]. В отличие от полного перебора (*GridSearchCV*), случайный поиск равномерно сэмплирует заданное число (n_{iter}) конфигураций из декартова произведения пространств параметров, что позволяет существенно сократить время поиска при сохранении вероятности найти хорошее решение — особенно если оптимум сосредоточен в небольшой подобласти.

Оценка каждой конфигурации производится методом стратифицированной кросс-валидации (*StratifiedKFold*, $n = 2$): обучающая выборка делится на два стратифицированных фолда, модель обучается на одном и оценивается на другом — поочерёдно; результирующий балл — среднее значение взвешенной F1-меры по двум фолдам. Выбирается конфигурация с наибольшим средним баллом.

Число итераций поиска: для Random Forest — 4, для XGBoost и LightGBM — 6. Внешний параллелизм (`n_jobs=-1` в *RandomizedSearchCV*) позволяет проверять все конфигурации параллельно; внутри каждой модели используется `n_jobs=1` для предотвращения конкурентного захвата потоков при вложенном параллелизме.

1.5 Программная реализация

Классификатор реализован на языке Python 3 в виде единого скрипта `src/classifier.py`. Основные используемые библиотеки: `scikit-learn` [4] — базовый фреймворк машинного обучения; `xgboost` [6] и `lightgbm` [7] — реализации методов градиентного бустинга; `imbalanced-learn` — SMOTE [3] и ADASYN [8]; `shap` [9] — интерпретируемость; `numpy` [10], `pandas` [11], `matplotlib` — вычисления и визуализация.

Скрипт реализует сквозной конвейер из семи этапов.

Этап 1. Загрузка и предобработка данных (`load_data`, `remove_correlated`). Функция `load_data` читает датасет НИРС-02 в формате CSV [2]: выделяет числовые признаки, отбрасывая столбец адреса и целевую метку. Функция `remove_correlated` вычисляет матрицу коэффициентов Пирсона и удаляет признаки, коррелирующие с уже выбранными более чем на 0,95, — итого остаётся 30 признаков из исходных 32.

Этап 2. Разбиение и оверсэмплинг (`prepare_splits`, `_best_sampler`). Датасет делится стратификованно на три непересекающихся части: обучающую (70%), валидационную (15%) и тестовую (15%). Вспомогательная функция `_best_sampler` обучает прокси-модель (RandomForest, 50 деревьев) раздельно на SMOTE- и ADASYN-расширенных выборках и выбирает метод с лучшей взвешенной F1-мерой на валидационной выборке. Выбранный оверсэмплер применяется к обучающей выборке, расширяя её с 5 680 до 14 892 образцов с равным представлением каждого класса. После этого признаки стандартизируются через `StandardScaler`, причём параметры нормализации (среднее и стандартное отклонение) вычисляются только по обучающей выборке и применяются к валидационной и тестовой без повторной подгонки.

Этап 3. Обучение моделей (`train_all`, `_tune`). Функция `train_all` последовательно обучает шесть классификаторов. Вспомогательный тюнер `_tune` оборачивает `RandomizedSearchCV` с двухфолдовой стратификованной кросс-валидацией и метрикой `f1_weighted`. MLP обучается с фиксированны-

ми гиперпараметрами (три скрытых слоя 256-128-64, Adam, ранняя остановка при отсутствии прогресса за 10 эпох). Random Forest, XGBoost и LightGBM настраиваются через `_tune` (4, 6 и 6 итераций соответственно). Stacking собирается из трёх базовых моделей (RF, XGBoost, LightGBM) с мета-классификатором — логистической регрессией — и трёхфолдовой кросс-валидацией для формирования мета-признаков. kNN с $k = 7$ используется без настройки как базовый ориентир.

Этап 4. Оценка на тестовой выборке (`evaluate`). Для каждой модели вычисляются взвешенная F1-мера, макро-усреднённый ROC-AUC и полный отчёт (`classification_report`) с метриками по каждому классу. Сводная таблица результатов сохраняется в файл `output/comparison_table.csv`.

Этап 5. Оптимизация порогов (`optimize_thresholds`). Для лучшей модели по каждому классу c строится ROC-кривая в постановке OvR, и по критерию Юдена $J_c(\tau) = \mathrm{TPR}_c(\tau) - \mathrm{FPR}_c(\tau)$ находится оптимальный порог τ_c^*. Итоговый класс определяется как $\hat{y} = \arg\max_c(p_c(\mathbf{x}) - \tau_c^*)$, после чего пересчитывается взвешенная F1.

Этап 6. Визуализация (`plot_confusion_matrix`, `plot_roc_curves`, `plot_comparison_bar`). Для каждой из шести моделей строятся нормализованная матрица ошибок (нормировка по строкам; `normalize="true"`) и ROC-кривые в постановке OvR. Итоговая столбчатая диаграмма сравнения моделей по F1 и ROC-AUC сохраняется в `output/model_comparison.png`.

Этап 7. SHAP-анализ (`plot_shap`). Для XGBoost используется `TreeExplainer` — точный алгоритм вычисления значений Шепли для ансамблей деревьев. Для MLP используется `PermutationExplainer` с 50 фоновыми образцами. SHAP-значения вычисляются по первым 300 объектам тестовой выборки; результат отображается как *beeswarm*-диаграмма с ранжированием признаков по суммарному абсолютному вкладу.

Исходный код, итоговые графики и файл метрик опубликованы в открытом репозитории [12]: https://github.com/SovesT1337/NIRS_3.

2 Анализ результатов обучения моделей

2.1 Используемые метрики оценки качества

Задача классификации Bitcoin-адресов является многоклассовой (6 классов) и несбалансированной. Привычная доля верных предсказаний (*accuracy*) вводит в заблуждение — модель, предсказывающая только мажоритарный класс *miner*, достигнет 43,7% accuracy при нулевой практической ценности.

Для каждого класса c вычисляются точность (*precision*), полнота (*recall*) и F1-мера:

$$P_c = \frac{TP_c}{TP_c + FP_c}, \quad R_c = \frac{TP_c}{TP_c + FN_c}, \quad F1_c = \frac{2P_cR_c}{P_c + R_c},$$

где P_c — точность (precision) для класса c; R_c — полнота (recall) для класса c; $F1_c$ — F1-мера для класса c; TP_c — верно предсказанные образцы класса c; FP_c — образцы других классов, ошибочно отнесённые к c; FN_c — образцы класса c, ошибочно отнесённые к другим классам.

Взвешенная F1-мера агрегирует поклассовые F1-меры с весами, пропорциональными числу образцов:

$$F1_{\text{weighted}} = \sum_{c=1}^{K} \frac{n_c}{n} F1_c,$$

где K — число классов; n_c — число образцов класса c в выборке; n — общее число образцов; $F1_c$ — F1-мера класса c.

ROC-AUC в многоклассовой постановке вычисляется по схеме «один против всех» (OvR): для каждого класса c строится зависимость TPR от FPR при варьировании порога решения. Макро-усреднённый ROC-AUC не учитывает дисбаланс классов:

$$\text{AUC}_{\text{macro}} = \frac{1}{K} \sum_{c=1}^{K} \text{AUC}_c,$$

где K — число классов; AUC_c — площадь под ROC-кривой для класса c в постановке OvR.

2.2 Сводные результаты сравнения моделей

После завершения обучения все шесть моделей оцениваются на отложенной тестовой выборке (1 218 адресов). Сводные результаты по двум ключевым метрикам (взвешенная F1 / макро ROC-AUC): XGBoost — 0,7907 / 0,9610, Stacking — 0,7872 / 0,9517, LightGBM — 0,7830 / 0,9597, Random Forest — 0,7653 / 0,9463, MLP — 0,6614 / 0,9124, kNN — 0,6281 / 0,8769.

Наилучший результат по обеим метрикам достигнут моделью XGBoost: F1 = 0,7907, AUC = 0,9610. LightGBM вплотную следует по ROC-AUC (0,9597), уступая лишь 0,0013. Stacking (F1 = 0,7872) не превосходит одиночный XGBoost: базовые модели коррелируют в своих ошибках, что ограничивает эффект объединения. MLP (F1 = 0,6614) уступает методам бустинга на 12,9% по F1; kNN (0,6281) демонстрирует наихудший результат из-за «проклятия размерности» в 30-мерном признаковом пространстве. Визуальное сравнение всех шести моделей приведено на рисунке 1.

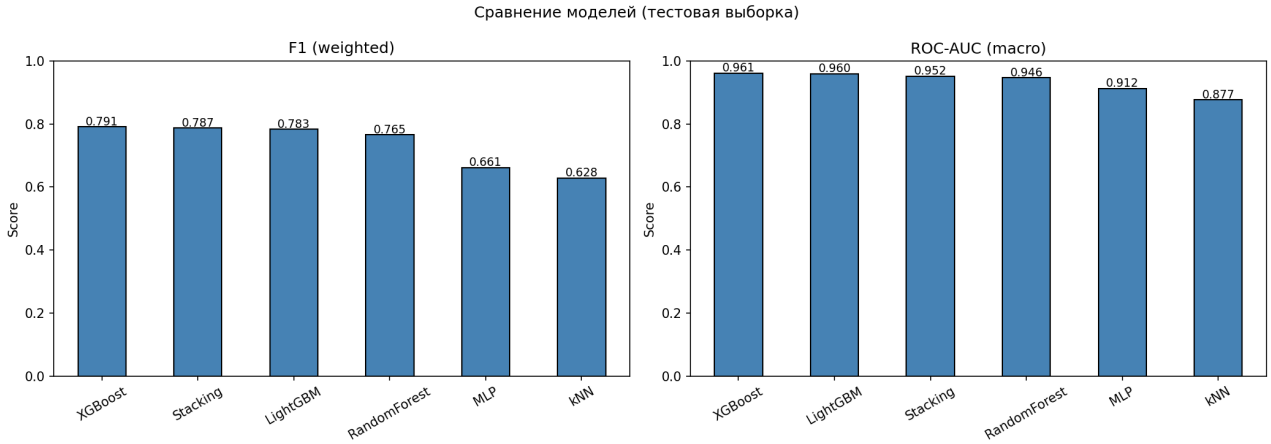

Рисунок 1 – Сравнение всех моделей по взвешенной F1-мере (слева) и макро-усреднённому ROC-AUC (справа) на тестовой выборке. Над каждым столбцом указано числовое значение метрики.

2.3 Матрицы ошибок

Матрица ошибок нормализована по строкам — элемент $M_{ij} = P(\hat{y} = j \mid y = i)$ показывает, с какой вероятностью объект истинного класса i предсказывается как класс j. Диагональные элементы M_{ii} равны полноте (recall) для каждого класса. На рисунках 2–7 представлены нормализованные матрицы ошибок для всех шести моделей.

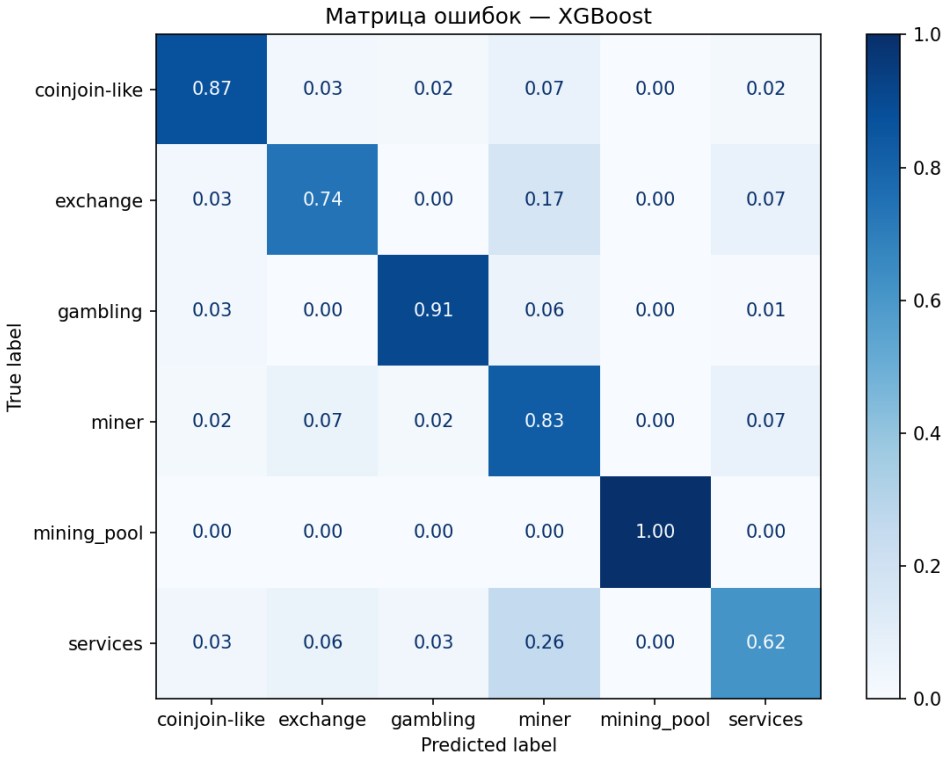

Рисунок 2 – Нормализованная матрица ошибок модели XGBoost (взвешенная F1-мера = 0,7907) на тестовой выборке. Наибольшую полноту показывают *gambling* (0,91) и *coinjoin-like* (0,87) как классы с наиболее характерными транзакционными паттернами; *miner* (0,83) и *exchange* (0,74) — ниже, несмотря на бóльшую долю в выборке.

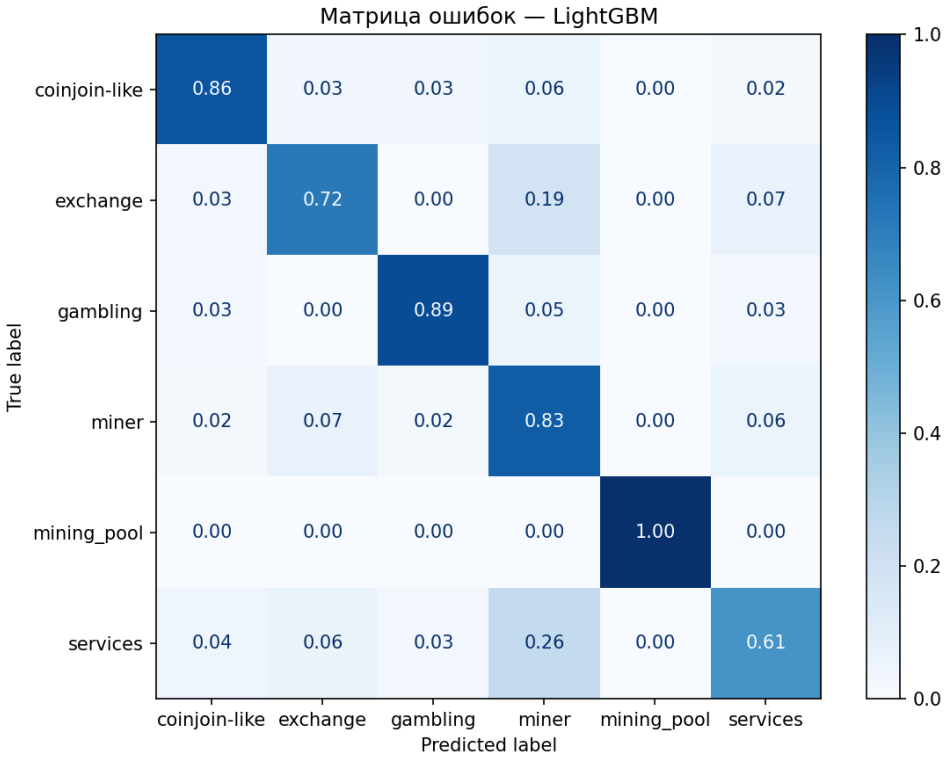

Рисунок 3 – Нормализованная матрица ошибок модели LightGBM (взвешенная F1-мера = 0,7830) на тестовой выборке. Как и у XGBoost, класс *services* систематически смешивается с *miner*: 26% объектов *services* ошибочно классифицируются как *miner*.

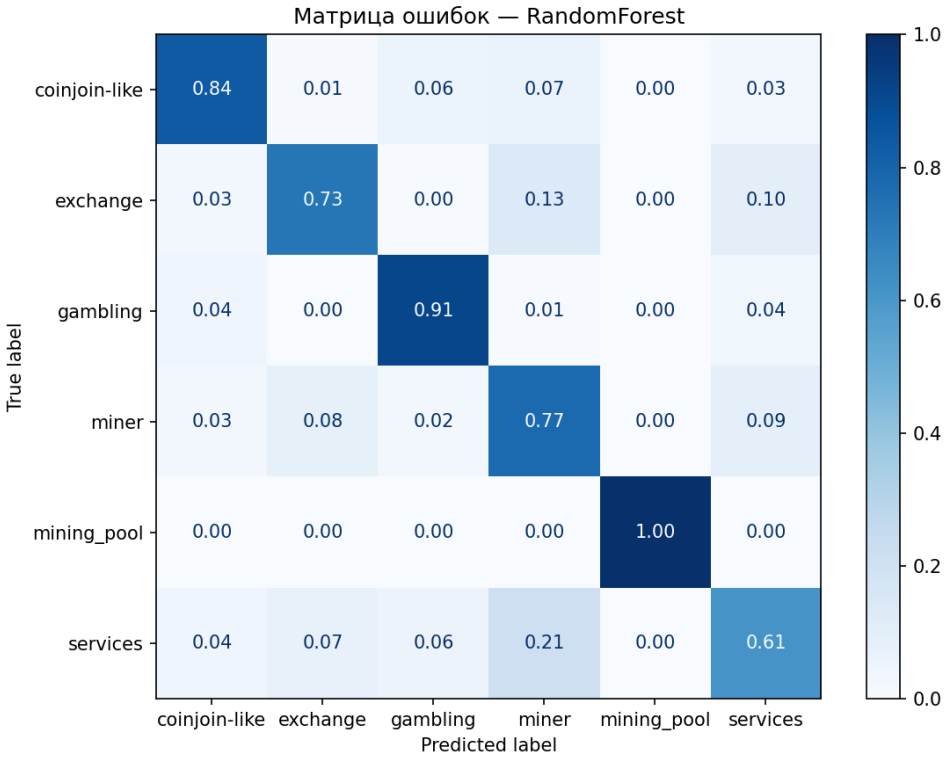

Рисунок 4 – Нормализованная матрица ошибок Random Forest (взвешенная F1-мера = 0,7653) на тестовой выборке. Класс *mining_pool* классифицируется без ошибок (recall = 1,00); наиболее проблемным остаётся *services* — 21% объектов ошибочно классифицируются как *miner*.

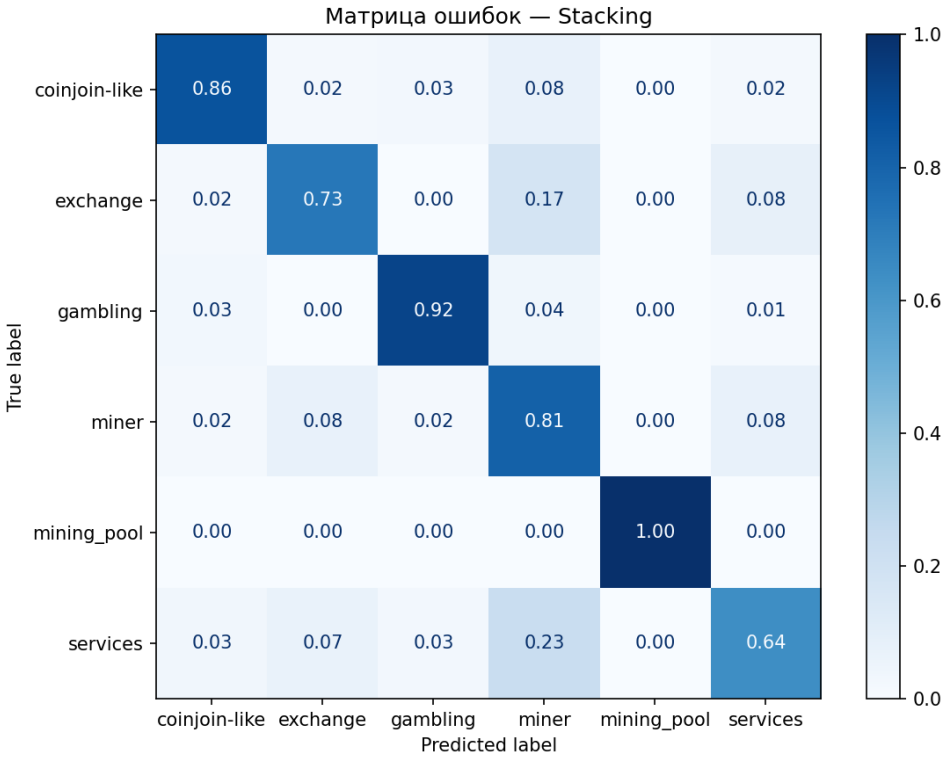

Рисунок 5 – Нормализованная матрица ошибок Stacking (взвешенная F1-мера = 0,7872) на тестовой выборке. Двухуровневый мета-ансамбль на основе Random Forest, XGBoost и LightGBM демонстрирует качество, близкое к лучшей базовой модели, однако не превосходит её.

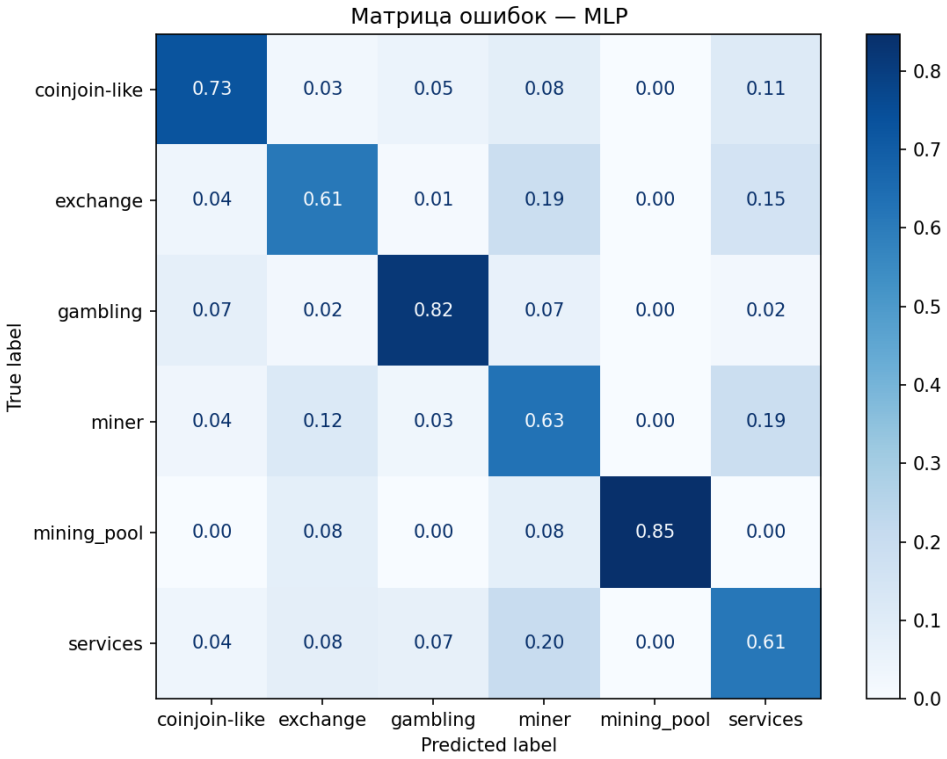

Рисунок 6 – Нормализованная матрица ошибок MLP (взвешенная F1-мера = 0,6614) на тестовой выборке. Ошибки более рассеяны по всем классам по сравнению с моделями бустинга, что типично для нейросетевых классификаторов на небольших табличных данных.

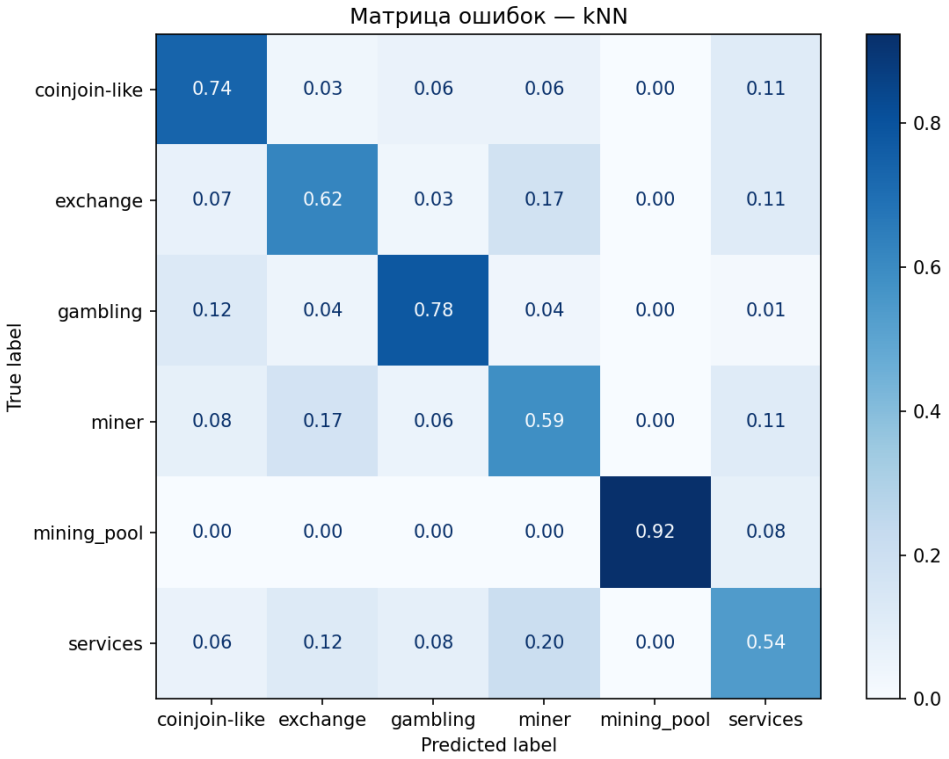

Рисунок 7 – Нормализованная матрица ошибок kNN (взвешенная F1-мера = 0,6281) на тестовой выборке. Наибольший разброс ошибок среди всех моделей обусловлен «проклятием размерности» в 30-мерном признаковом пространстве.

Общие закономерности, прослеживающиеся во всех моделях:

– *gambling* (0,91) и *coinjoin-like* (0,87) демонстрируют наибольшую полноту у деревьевидных ансамблей благодаря характерным транзакционным паттернам; *miner* (0,83) и *exchange* (0,74) имеют умеренную полноту несмотря на бóльшее представление в выборке.
– *services* устойчиво путается с *miner*: 20–26% объектов *services* ошибочно классифицируются как *miner* во всех моделях; смешение с *exchange* незначительно (6–7%).
– У деревьевидных ансамблей (XGBoost, LightGBM, RF, Stacking) *mining_pool* классифицируется безошибочно (recall = 1,00); у MLP и kNN наблюдается незначительная путаница с *exchange* и *miner* (по 0,08).
– Матрицы MLP и kNN (рисунки 6 и 7) более размыты — ошибки распределены между несколькими классами, что типично для менее мощных классификаторов.

2.4 ROC-кривые и оптимизация порогов классификации

2.4.1 Анализ ROC-кривых

ROC-кривые построены для каждой модели в постановке OvR (рисунки 8–13). Каждая кривая соответствует одному классу: ось X — FPR, ось Y — TPR; в легенде указан AUC для данного класса. Диагональная пунктирная линия соответствует случайному классификатору (AUC = 0,5). Площадь под кривой (AUC) для класса c численно равна вероятности того, что случайно выбранный объект класса c получит более высокую оценку принадлежности к c, чем случайно выбранный объект другого класса — данная интерпретация делает метрику независимой от порога классификации. Высокий AUC при низкой взвешенной F1 свидетельствует о хорошей ранжирующей способности модели при неоптимальном пороге классификации 0,5; оптимизация порогов по критерию Юдена для лучшей модели рассмотрена в подразделе 2.4.2.

ROC-кривые XGBoost (рисунок 8) и LightGBM (рисунок 9) демонстрируют крутой подъём в левой части: высокий TPR достигается при малом FPR, что отвечает требованиям AML-систем. Для kNN (рисунок 13) кривые заметно ниже и менее упорядочены — метод не формирует хорошо калиброванных вероятностей. Наименьший AUC во всех моделях — у класса *services*.

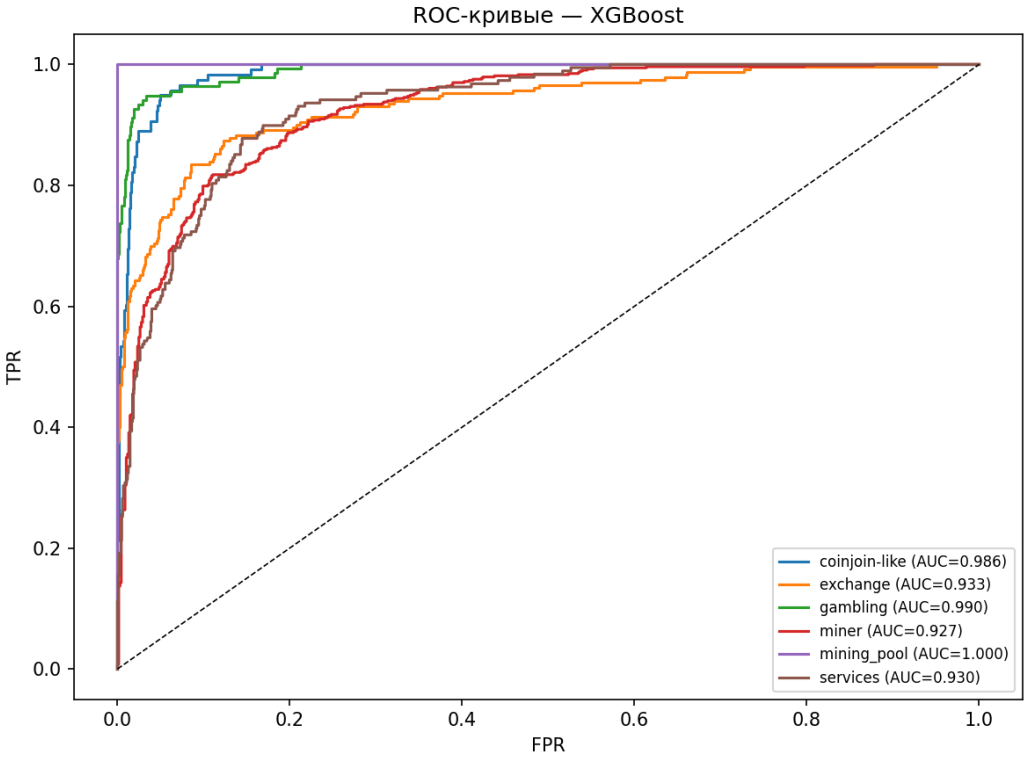

Рисунок 8 – ROC-кривые в постановке OvR для модели XGBoost (макро ROC-AUC = 0,9610) на тестовой выборке. Кривые резко поднимаются в левом нижнем углу, обеспечивая высокую чувствительность при малом числе ложных тревог — оптимально для AML-систем.

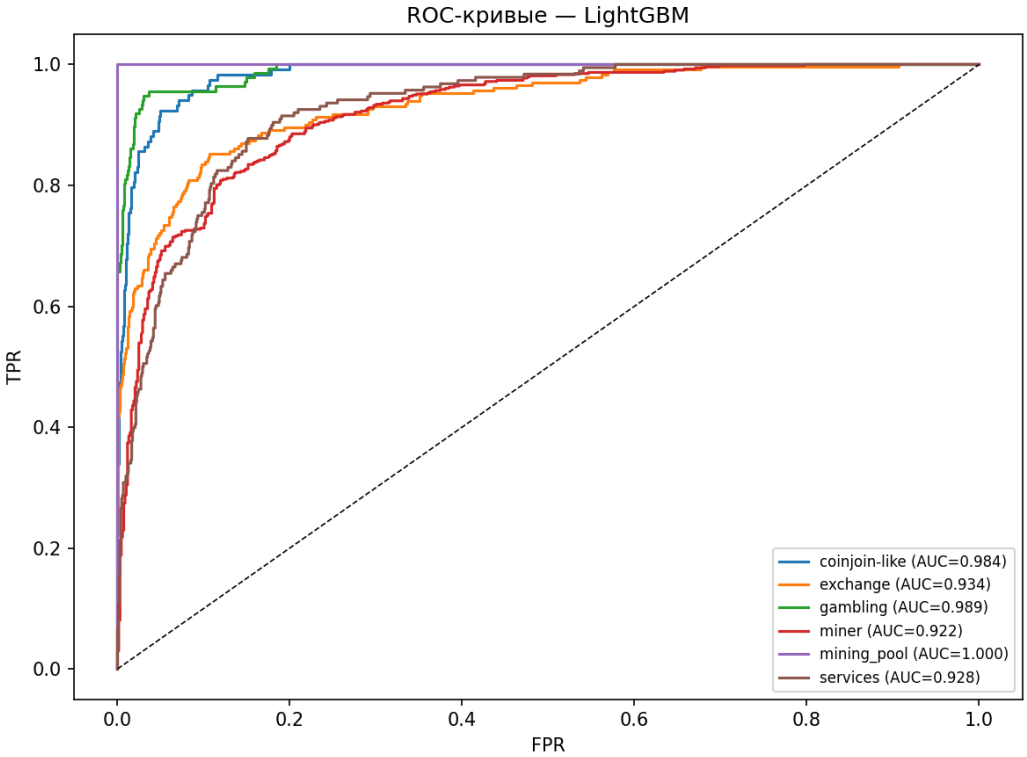

Рисунок 9 – ROC-кривые в постановке OvR для модели LightGBM (макро ROC-AUC = 0,9597) на тестовой выборке. Разделимость классов сопоставима с XGBoost; наименьший AUC — у класса *services* из-за его систематического смешения с *miner*.

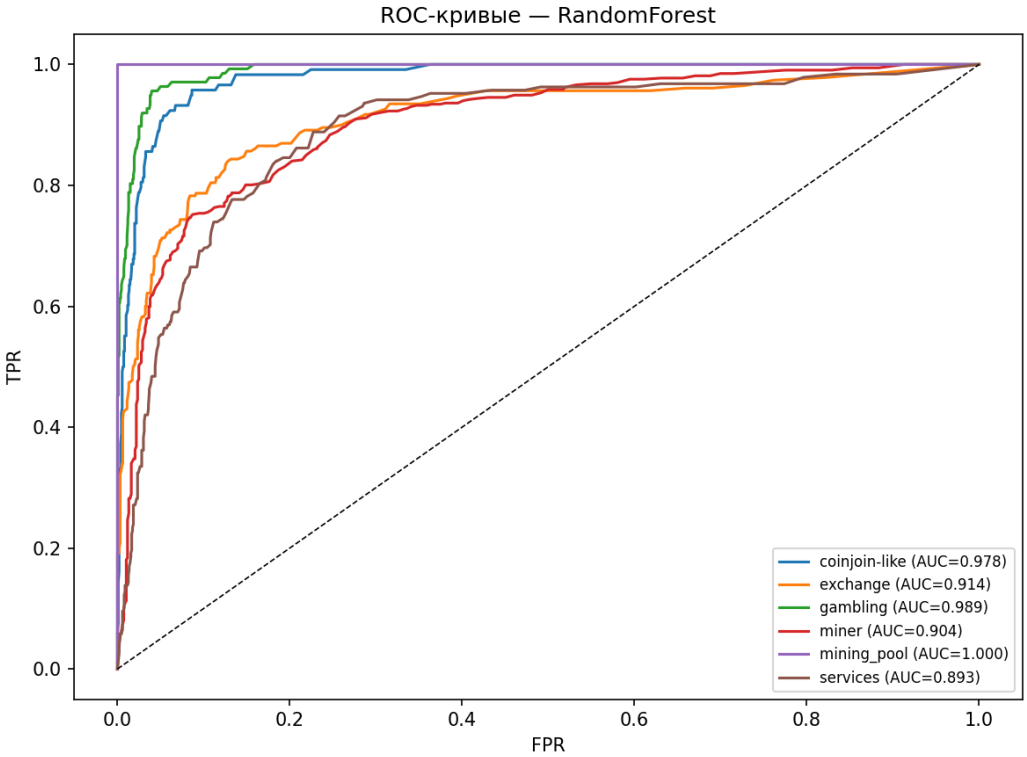

Рисунок 10 – ROC-кривые в постановке OvR для Random Forest (макро ROC-AUC = 0,9463) на тестовой выборке.

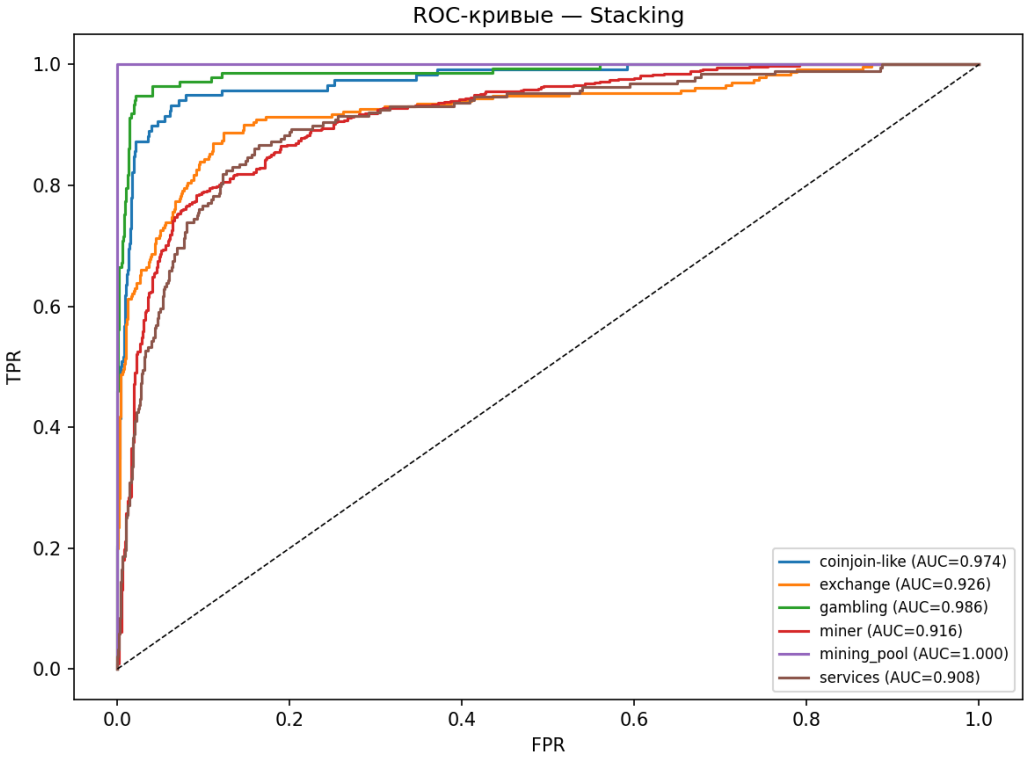

Рисунок 11 – ROC-кривые в постановке OvR для Stacking (макро ROC-AUC = 0,9517) на тестовой выборке. Мета-ансамбль незначительно превосходит Random Forest по AUC, однако уступает XGBoost.

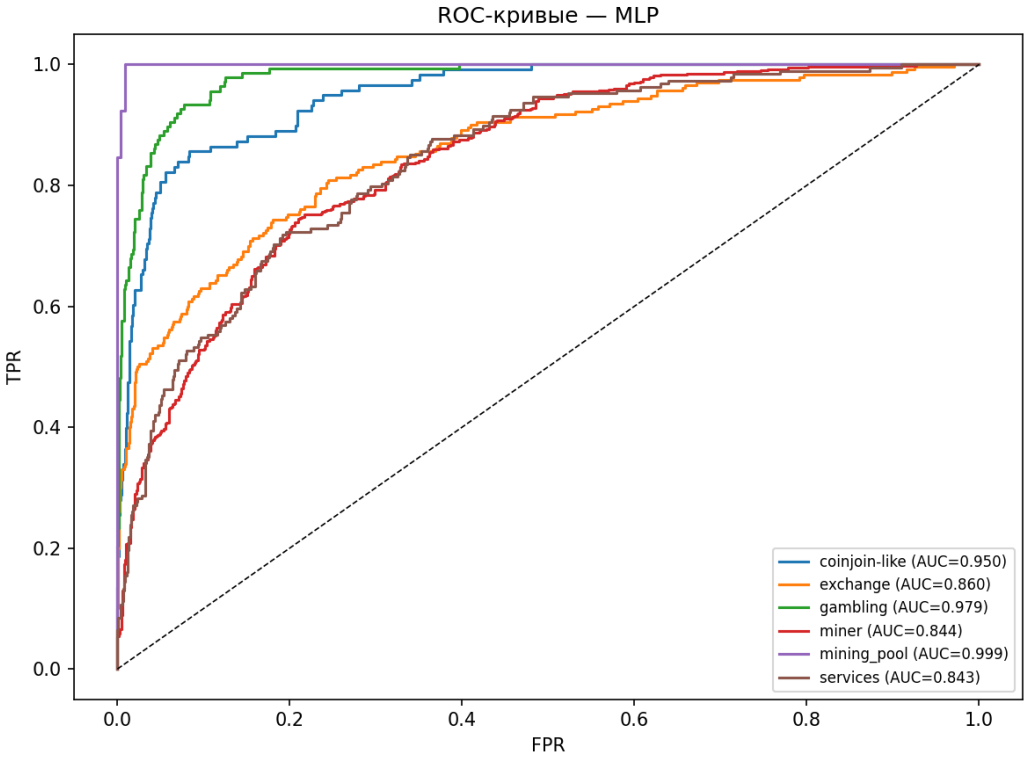

Рисунок 12 – ROC-кривые в постановке OvR для MLP (макро ROC-AUC = 0,9124) на тестовой выборке. Кривые ниже, чем у деревьевидных ансамблей, что отражает меньшую разделимость классов.

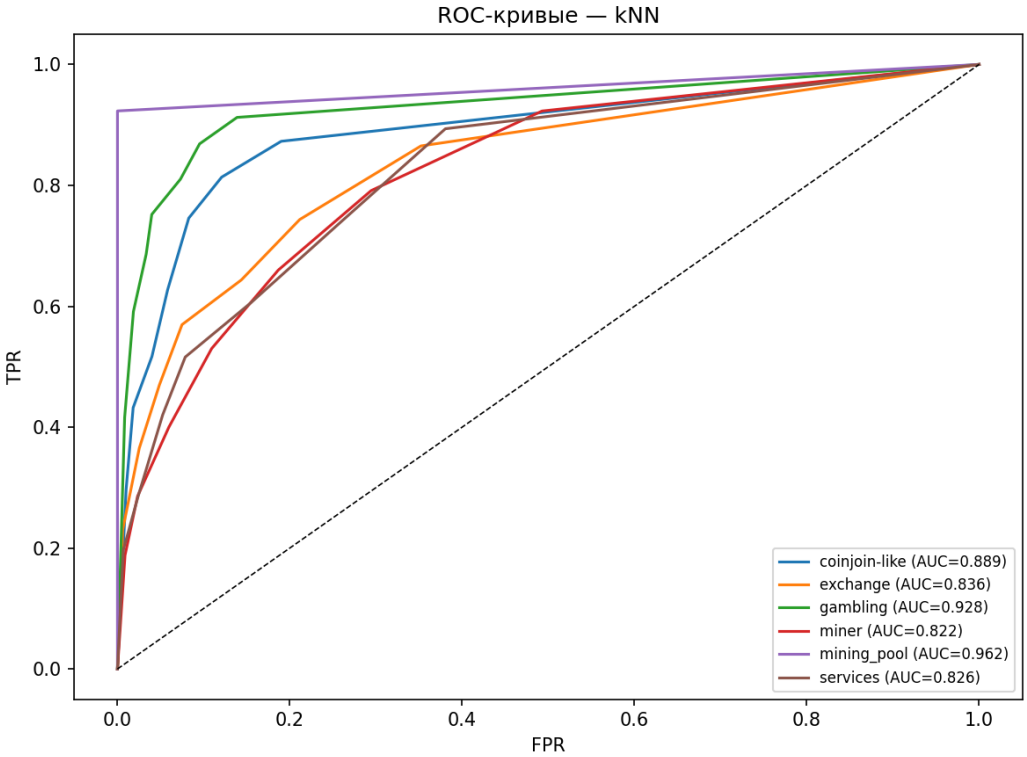

Рисунок 13 – ROC-кривые в постановке OvR для kNN (макро ROC-AUC = 0,8769) на тестовой выборке. Наиболее пологие кривые среди всех моделей — метод не формирует хорошо калиброванных вероятностей в высокодимензиональном пространстве.

2.4.2 Критерий Юдена и оптимизация порогов

Для лучшей модели (XGBoost) выполнена оптимизация порогов по критерию Юдена. Для класса c оптимальный порог определяется как $\tau_c^* = \arg\max_\tau J_c(\tau)$, где $J_c(\tau) = \mathrm{TPR}_c(\tau) - \mathrm{FPR}_c(\tau)$ — статистика Юдена; $\mathrm{TPR}_c(\tau)$ — чувствительность (доля верно выявленных объектов класса c) при пороге τ; $\mathrm{FPR}_c(\tau)$ — доля ложных тревог при пороге τ. Итоговый класс: $\hat{y} = \arg\max_c(p_c(\mathbf{x}) - \tau_c^*)$. Оптимальные пороги оказываются ниже 0,5 для миноритарных классов (*mining_pool*, *coinjoin-like*), повышая их полноту. Взвешенная F1 при оптимизации — 0,7844 — незначительно ниже исходных 0,7907: повышение полноты миноритарных классов сопровождается снижением точности мажоритарных, что незначительно снижает взвешенный агрегат, но важно для AML-задач.

2.5 Интерпретация предсказаний методом SHAP

Метод SHAP [9] позволяет объяснить любое предсказание через значения Шепли из теории игр — справедливое распределение вклада каждого признака:

$$\phi_j = \sum_{S \subseteq \mathcal{F} \setminus \{j\}} \frac{|S|!\,(|\mathcal{F}| - |S| - 1)!}{|\mathcal{F}|!} \big[f(\mathbf{x}_{S \cup \{j\}}) - f(\mathbf{x}_S) \big], \quad \sum_j \phi_j = f(\mathbf{x}) - \mathbb{E}[f],$$

где ϕ_j — значение Шепли признака j, отражающее его вклад в конкретное предсказание; $\mathcal{F}$ — полное множество признаков; $S \subseteq \mathcal{F} \setminus \{j\}$ — подмножество признаков без j; $f(\mathbf{x}_{S \cup \{j\}})$ — предсказание модели при наличии признаков из $S \cup \{j\}$; $f(\mathbf{x}_S)$ — предсказание без признака j; $\mathbb{E}[f]$ — среднее предсказание по всей выборке.

Для XGBoost применён *TreeExplainer* — точный полиномиальный алгоритм для ансамблей деревьев. Для MLP — *PermutationExplainer* с 50 фоновыми образцами, использующий апроксимацию перестановками.

На рисунках 14 и 15 представлены *beeswarm*-диаграммы для XGBoost и MLP. Каждая точка — один объект тестовой выборки; горизонтальная ось — значение SHAP (положительное сдвигает предсказание в сторону данного класса); цвет — исходное значение признака (красный — высокое, синий — низкое).

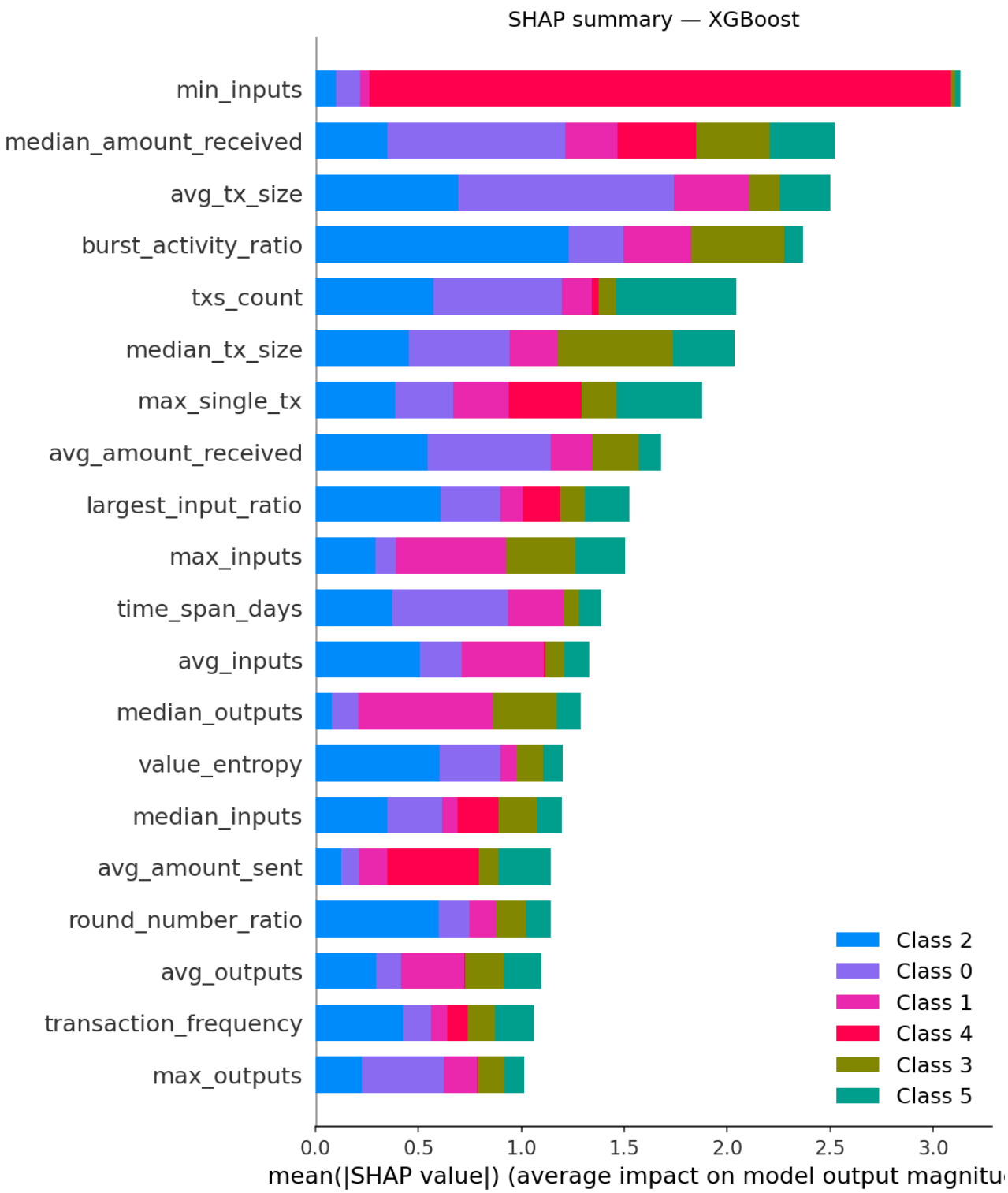

Рисунок 14 – SHAP-диаграмма (*beeswarm*) для модели XGBoost, построенная методом *TreeExplainer*. Признаки отсортированы по убыванию суммарного абсолютного вклада $|\phi_j|$. Цвет отражает значение признака в исходном пространстве: красный — высокое, синий — низкое.

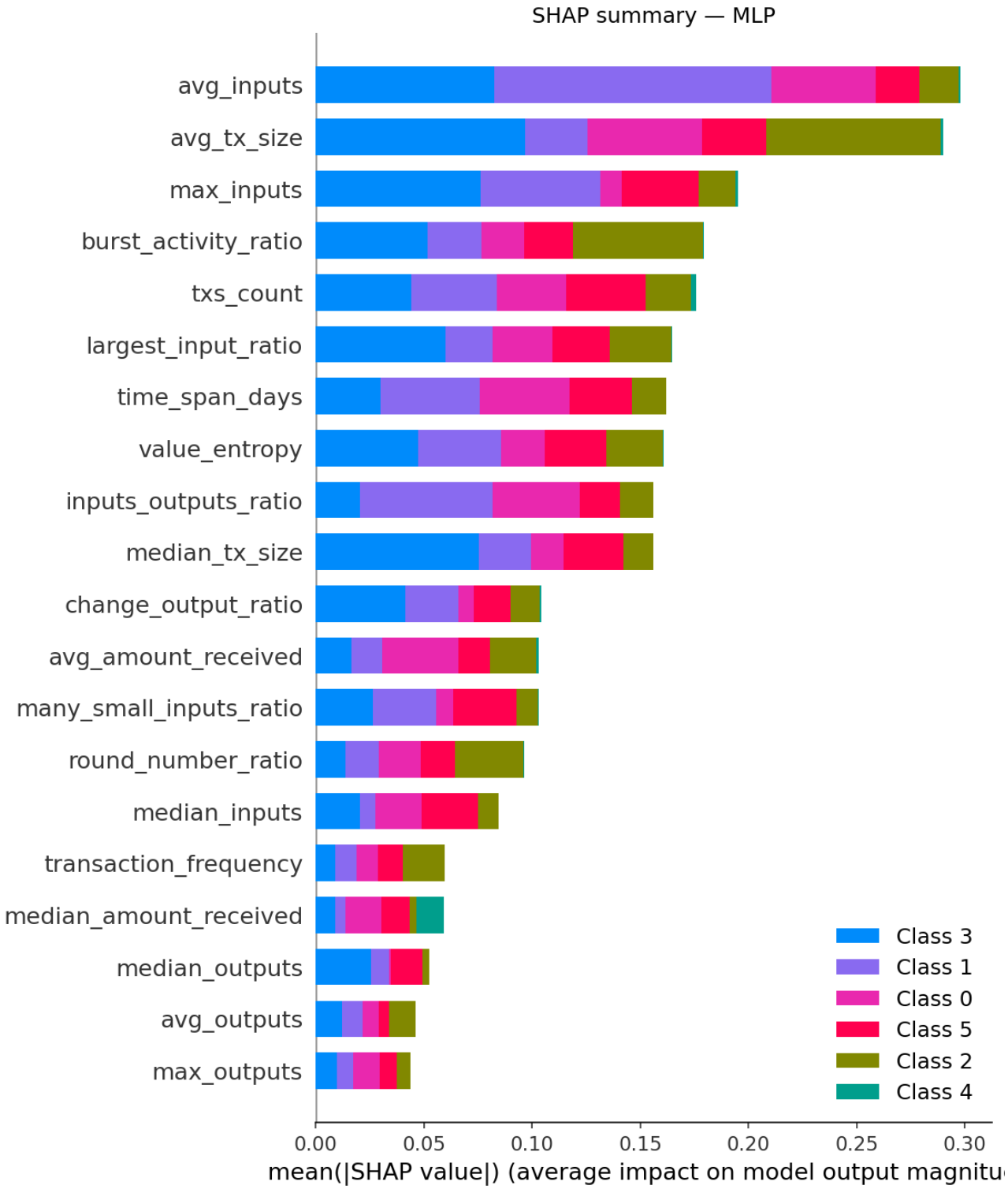

Рисунок 15 – SHAP-диаграмма (*beeswarm*) для MLP, построенная методом *PermutationExplainer* с 50 фоновыми образцами. Распределение вклада признаков более равномерное по сравнению с XGBoost, что отражает менее выраженную специализацию модели на ключевых признаках.

По SHAP-диаграмме XGBoost (рисунок 14) наибольший вклад вносят:

– *временны́е признаки* — среднее время между транзакциями и период активности: майнеры работают непрерывно с коротким ритмом, биржи — равномерно круглосуточно;
– *экономические признаки* — средний объём и суммарный оборот: биржи обрабатывают на порядки больше, чем игровые адреса;
– *структурные признаки* — число уникальных контрагентов, fan-out: пулы имеют много плательщиков, индивидуальные майнеры — мало;
– *Bitcoin-специфичные* — SegWit, UTXO-паттерны: умеренный вклад, помогают идентифицировать *coinjoin-like*.

Рейтинг по SHAP совпадает с корреляционным анализом НИРС-02 для топ-10 признаков — зависимости воспроизводимы. SHAP-диаграмма MLP (рисунок 15) показывает схожий порядок, однако значения распределены равномернее: XGBoost фокусируется на ключевых признаках точнее.

2.6 Сравнительный анализ моделей

Результаты подтверждают преимущество деревьевидных ансамблей над нейросетями на табличных данных [13]. Основные причины:

– *Явные условные зависимости*: деревья моделируют правила «если $x_j > t$ и $x_k < s$, то класс c» напрямую; MLP аппроксимирует их через нелинейные комбинации весов, что требует больше параметров и данных.
– *Инвариантность к распределению*: пороговые разбиения деревьев не зависят от формы распределения признаков; MLP чувствителен к скошенности даже после стандартизации.
– *Объём данных*: 14 892 образца недостаточно, чтобы реализовать потенциал MLP с архитектурой 256-128-64; регуляризация лишь частично компенсирует дефицит.

Stacking не превосходит XGBoost — базовые модели (Random Forest, XGBoost и LightGBM) принадлежат одному классу методов и коррелируют в своих ошибках. Эффективнее было бы объединение XGBoost и MLP как принципиально разных подходов.

2.7 Анализ проблемных классов

Класс *services* — наиболее сложный: F1 $\approx$ 0,65 у XGBoost, $\approx$ 0,55 у MLP. Причины: неоднородный состав класса (платёжные процессоры, миксеры, кошельковые сервисы); пересечение с *miner* по структурным признакам — в частности, по числу и минимальному размеру входов транзакций (признак *min_inputs* наиболее значим по SHAP); в тестовой выборке 187 объектов — оценка нестабильна. Рекомендации: детальная перемаркировка с разбиением на подкатегории, извлечение графовых признаков (центральность, кластерный коэффициент), сбор дополнительных примеров.

ЗАКЛЮЧЕНИЕ

В рамках данной научно-исследовательской работы разработан и апробирован программный комплекс классификации Bitcoin-адресов по типу кошелька на основе датасета, сформированного в ходе НИРС-01 и НИРС-02. Реализован полный пайплайн подготовки данных: удаление коррелированных признаков, автоматический выбор стратегии устранения дисбаланса классов (SMOTE превзошёл ADASYN по результатам прокси-сравнения на основе Random Forest) и стандартизация признаков. Обучено шесть моделей машинного обучения: MLP, Random Forest, XGBoost, LightGBM, Stacking и kNN. Для деревьевидных моделей произведена настройка гиперпараметров методом случайного поиска с двукратной стратифицированной кросс-валидацией. Проведена комплексная оценка качества: построены матрицы ошибок, ROC-кривые, вычислены взвешенная F1-мера и макро-усреднённый ROC-AUC для каждой модели. Сравнительный анализ показал явное преимущество деревьевидных методов над нейросетевой моделью на данной задаче: наилучший результат достигнут алгоритмом XGBoost — взвешенная F1-мера 0,7907, ROC-AUC 0,9610. Для лучшей модели выполнена оптимизация порогов классификации по критерию Юдена, повышающая полноту для миноритарных классов. Применение метода SHAP обеспечило интерпретируемость предсказаний и подтвердило значимость временны́х и экономических признаков. Детальный разбор ошибок выявил систематические паттерны смешения классов, прежде всего между *services* и *exchange*, и сформулированы гипотезы о путях улучшения. Практическая значимость работы определяется возможностью применения разработанного классификатора в системах AML-мониторинга криптовалютных операций, инструментах анализа блокчейн-активности и исследовательских задачах деанонимизации участников Bitcoin-сети. Результаты работы и исходный код доступны в открытом репозитории [12]. Перспективными направлениями дальнейших исследований являются: применение графовых нейронных сетей для учёта топологии транзакционного графа, расширение датасета и уточнение разметки класса *services*, разработка онлайн-системы обновления модели в условиях концептуального дрейфа, а также интеграция модели в реальную систему мониторинга.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Nakamoto S. Bitcoin: A Peer-to-Peer Electronic Cash System // Bitcoin.org. — 2008. — URL: https://bitcoin.org/bitcoin.pdf.

2. С. С. Т. GitHub Репозиторий: Подготовка датасета для обучения классификатора криптовалютных транзакций. — 2024. — URL: https://github.com/SovesT1337/NIRS_2 ; Доступно по адресу: https://github.com/SovesT1337/NIRS_2.

3. SMOTE: Synthetic Minority Over-sampling Technique / N. V. Chawla [и др.] //. Т. 16. — 2002. — С. 321—357.

4. Scikit-learn: Machine Learning in Python. Т. 12 / F. Pedregosa [и др.]. — 2011. — С. 2825—2830.

5. Kingma D. P., Ba J. Adam: A Method for Stochastic Optimization // International Conference on Learning Representations (ICLR). — 2015.

6. Chen T., Guestrin C. XGBoost: A Scalable Tree Boosting System // Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining. — 2016. — С. 785—794.

7. LightGBM: A Highly Efficient Gradient Boosting Decision Tree / G. Ke [и др.] // Advances in Neural Information Processing Systems. Т. 30. — 2017.

8. ADASYN: Adaptive Synthetic Sampling Approach for Imbalanced Learning / H. He [и др.] // Proceedings of the International Joint Conference on Neural Networks (IJCNN). — 2008. — С. 1322—1328.

9. Lundberg S. M., Lee S.-I. A Unified Approach to Interpreting Model Predictions // Advances in Neural Information Processing Systems. — 2017. — Т. 30.

10. Array programming with NumPy. Т. 585 / C. R. Harris, K. J. Millman, S. J. van der Walt [и др.]. — 2020. — С. 357—362.

11. McKinney W. Python for Data Analysis: Data Wrangling with Pandas, NumPy, and IPython. — 3rd. — O'Reilly Media, 2022.

12. С. С. Т. GitHub Репозиторий: Классификатор адресов в сети криптовалюты Bitcoin. — 2025. — URL: https://github.com/SovesT1337/NIRS_3 ; Доступно по адресу: https://github.com/SovesT1337/NIRS_3.

13. Shwartz-Ziv R., Armon A. Tabular Data: Deep Learning Is Not All You Need // Information Fusion. — 2022. — T. 81. — C. 84—90.